Department of Computing

National University of Sciences and Technology (NUST)

CS332 – Parallel & Distributed Computing

LAB 07

Synchronization & Election Algorithm

Bully Algorithm & Ring Algorithm

Student	Abrar Butt
QALAM ID	460297
Instructor	Dr. Shah Khalid
Class	BSCS
Course	CS332

GitHub Repository:

https://github.com/Abrarbut/parallel-and-computing-distribution.git

Contents

1 Introduction **2**

2 Objectives **2**

3 Tools / Software **2**

4 Description **2**

4.1 Election Algorithms – Overview 2

4.2 The Bully Algorithm 3

4.3 The Ring Algorithm 3

5 Lab Tasks **5**

5.1 Task 1 – Bully Algorithm 5

5.2 Task 2 – Ring Algorithm 10

6 Code Files **14**

7 Deliverables **14**

1. Introduction

A **Distributed Algorithm** is an algorithm that runs on a distributed system – a collection of independent computers that do not share memory. Each processor has its own memory and they communicate via communication networks; a process on one machine sends messages to a process on another.

Many algorithms used in distributed systems require a **coordinator** that performs functions needed by other processes. **Election algorithms** are designed to choose such a coordinator. When the current coordinator crashes, a new one is elected among the remaining active processes.

2. Objectives

- To understand the concept of **synchronization** in distributed systems.
- To learn and implement the **Bully Algorithm** for coordinator election.
- To learn and implement the **Ring Algorithm** for coordinator election.

3. Tools / Software

- **Language:** C / Java / C++ (choice of student)
- **IDE:** Microsoft Visual Studio / Eclipse

Useful Tutorial:
https://www.youtube.com/watch?v=vKpc0y_Ik3E (Lastmomenttuitions – Election Algorithms)

Reference Implementation:
http://www.c4learn.com/c-programs/c-program-for-bully-election-algorithm/

4. Description

4.1 Election Algorithms – Overview

Election algorithms choose a process from a group of processors to act as a **coordinator**. If the coordinator process crashes, a new coordinator is elected.

- Every active process has a **unique priority number**.
- The process with the **highest priority** becomes the new coordinator.
- When a coordinator fails, the algorithm elects the active process with the highest priority number and broadcasts its ID to every active process.

Two algorithms exist for different network configurations:

Algorithm	Applicable Configuration
Bully Algorithm	Every process can send a message to every other process (*fully connected*).
Ring Algorithm	Processes are arranged in a logical or physical ring; messages travel in one direction only.

4.2 The Bully Algorithm

This algorithm applies to systems where **every process can communicate with every other process**.

Steps

Suppose process P sends a message to the coordinator and receives no reply within time interval T:

Step 1. Process P sends an ELECTION message to all processes with a *higher* priority number.

Step 2. P waits for responses.

- If **no response** is received within T, P elects itself as coordinator and sends a COORDINATOR message to all lower-priority processes.
- If an **answer is received** from process Q within T:
  - (i) P waits for an additional interval T' to receive a COORDINATOR message from Q.
  - (ii) If Q does not send COORDINATOR within T', it is assumed to have failed and the algorithm restarts.

Key property: The process that "bullies" all lower-priority processes wins – hence the name *Bully Algorithm*. The process with the highest active priority always wins.

4.3 The Ring Algorithm

This algorithm applies to systems organized as a **ring** (logical or physical). Links are *unidirectional*: each process can only send messages to its right neighbour. The key data structure is an **active list** – a list of priority numbers of all active processes seen so far.

Steps

Step 1. Process P_1 detects coordinator failure. It creates an empty active list, adds its own ID (1) to it, and forwards an ELECTION message to its right neighbour.

Step 2. When a process P_2 receives an ELECTION message, it responds in one of three ways:

(i) If the active list does *not* contain P_2's ID, P_2 appends its own ID and forwards the message.

(ii) If this is the *first* election message P_2 has seen, it creates a new active list with both IDs and sends an updated ELECTION message.

(iii) If P_1 receives its *own* ELECTION message back, the active list now contains all active process IDs. P_1 selects the highest-priority ID and broadcasts a COORDINATOR message.

5. Lab Tasks

5.1 Task 1 – Bully Algorithm

Requirements:

- Number of processes: **7**
- Assign unique priorities to all processes.
- Display process state: `active -> 1` / `inactive -> 0`
- Assume **process 3** initiates the election.
- Select the leader/coordinator.
- Assume the chosen coordinator **fails**; repeat election among the 6 remaining processes.

Code: bully_algorithm.cpp

```cpp
#include <iostream>
using namespace std;

#define MAX_PROCESSES 7

struct Process {
    int  id;
    int  priority;
    bool active;   // true = active (1), false = inactive (0)
};

// Global array of processes
Process procs[MAX_PROCESSES];

//       Initialize 7 processes with unique priorities

void initProcesses() {
    int priorities[MAX_PROCESSES] = {4, 7, 2, 9, 1, 6, 3};
    for (int i = 0; i < MAX_PROCESSES; i++) {
        procs[i].id       = i + 1;          // IDs: 1..7
        procs[i].priority = priorities[i];
        procs[i].active   = true;
    }
}

//       Print current state of all processes

void printState() {
    cout << "\nProcess States:" << endl;
    cout << "+---------+----------+--------+" << endl;
    cout << "| Process | Priority | Status |" << endl;
    cout << "+---------+----------+--------+" << endl;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        cout << "|   P" << procs[i].id
```

```
             << "    |    " << procs[i].priority
             << "     |   " << (procs[i].active ? "1" : "0")
             << "    |" << endl;
    }
    cout << "+---------+----------+--------+" << endl;
}

//       Run Bully Election from initiator processIndex

int bullyElection(int initiatorIndex) {
    cout << "\n>> Process P" << procs[initiatorIndex].id
         << " (priority=" << procs[initiatorIndex].priority
         << ") initiates election." << endl;

    int highestPriority = -1;
    int coordinatorIdx  = -1;

    // Send ELECTION to all active processes with higher priority
    bool higherExists = false;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (procs[i].active &&
            procs[i].priority > procs[initiatorIndex].priority) {
            cout << "   P" << procs[initiatorIndex].id
                 << " sends ELECTION to P" << procs[i].id << endl;
            higherExists = true;
            if (procs[i].priority > highestPriority) {
                highestPriority = procs[i].priority;
                coordinatorIdx  = i;
            }
        }
    }

    if (!higherExists) {
        // No higher-priority process -> initiator wins
        coordinatorIdx = initiatorIndex;
        cout << "   No response. P" << procs[coordinatorIdx].id
             << " elects itself." << endl;
    } else {
        cout << "   P" << procs[coordinatorIdx].id
             << " (priority=" << highestPriority
             << ") wins and becomes coordinator." << endl;
    }

    // Broadcast COORDINATOR message
    cout << "\n>> P" << procs[coordinatorIdx].id
         << " broadcasts COORDINATOR message." << endl;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (procs[i].active && i != coordinatorIdx)
            cout << "   P" << procs[coordinatorIdx].id
                 << " -> P" << procs[i].id << " : COORDINATOR" << endl;
    }

    cout << "\n==> COORDINATOR elected: P"
         << procs[coordinatorIdx].id
         << " (priority=" << procs[coordinatorIdx].priority << ")"
         << endl;

    return coordinatorIdx;
```

```
}

int main() {
    cout << "=========================================" << endl;
    cout << "  Bully Algorithm -- 7 Processes          " << endl;
    cout << "=========================================" << endl;

    initProcesses();
    printState();

    //      Round 1: Process 3 (index 2) initiates election
    cout << "\n--- ROUND 1: Process P3 initiates election ---" << endl;
    int coordinator = bullyElection(2);   // P3 = index 2

    //      Coordinator fails

    cout << "\n>> Coordinator P" << procs[coordinator].id
         << " has FAILED. Setting inactive." << endl;
    procs[coordinator].active = false;

    printState();

    //      Round 2: P3 initiates again among 6 remaining
    cout << "\n--- ROUND 2: Process P3 initiates election "
         << "(6 remaining processes) ---" << endl;
    bullyElection(2);

    cout << "\n=========================================" << endl;
    cout << "  Simulation Complete" << endl;
    cout << "=========================================" << endl;

    return 0;
}
```

Listing 1: Bully Algorithm – 7 Processes

Output: Bully Algorithm

```
==========================================
  Bully Algorithm -- 7 Processes
==========================================

Process States:
+---------+----------+--------+
| Process | Priority | Status |
+---------+----------+--------+
|   P1    |    4     |   1    |
|   P2    |    7     |   1    |
|   P3    |    2     |   1    |
|   P4    |    9     |   1    |
|   P5    |    1     |   1    |
|   P6    |    6     |   1    |
|   P7    |    3     |   1    |
+---------+----------+--------+

--- ROUND 1: Process P3 initiates election ---

>> Process P3 (priority=2) initiates election.
   P3 sends ELECTION to P1
   P3 sends ELECTION to P2
   P3 sends ELECTION to P4
   P3 sends ELECTION to P6
   P3 sends ELECTION to P7
   P4 (priority=9) wins and becomes coordinator.

>> P4 broadcasts COORDINATOR message.
   P4 -> P1 : COORDINATOR
   P4 -> P2 : COORDINATOR
   P4 -> P3 : COORDINATOR
   P4 -> P5 : COORDINATOR
   P4 -> P6 : COORDINATOR
   P4 -> P7 : COORDINATOR

==> COORDINATOR elected: P4 (priority=9)

>> Coordinator P4 has FAILED. Setting inactive.

Process States:
+---------+----------+--------+
| Process | Priority | Status |
+---------+----------+--------+
|   P1    |    4     |   1    |
|   P2    |    7     |   1    |
|   P3    |    2     |   1    |
|   P4    |    9     |   0    |
|   P5    |    1     |   1    |
|   P6    |    6     |   1    |
|   P7    |    3     |   1    |
+---------+----------+--------+

--- ROUND 2: Process P3 initiates election (6 remaining processes) ---

>> Process P3 (priority=2) initiates election.
   P3 sends ELECTION to P1
```

```
   P3 sends ELECTION to P2
   P3 sends ELECTION to P6
   P3 sends ELECTION to P7
   P2 (priority=7) wins and becomes coordinator.

>> P2 broadcasts COORDINATOR message.
   P2 -> P1 : COORDINATOR
   P2 -> P3 : COORDINATOR
   P2 -> P5 : COORDINATOR
   P2 -> P6 : COORDINATOR
   P2 -> P7 : COORDINATOR

==> COORDINATOR elected: P2 (priority=7)

==========================================
  Simulation Complete
==========================================
```

Listing 2: Bully Algorithm – Actual Output

5.2 Task 2 – Ring Algorithm

Requirements:

- Number of processes: **7**, arranged in a ring.
- Assign unique priorities to all processes.
- Display process state: `active -> 1` / `inactive -> 0`
- Assume **process 3** initiates the election.
- Select the leader/coordinator.
- Assume the chosen coordinator **fails**; repeat election among the 6 remaining processes.

Code: ring_algorithm.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

#define MAX_PROCESSES 7

struct Process {
    int  id;
    int  priority;
    bool active;
};

Process procs[MAX_PROCESSES];

//       Initialize 7 processes

void initProcesses() {
    int priorities[MAX_PROCESSES] = {4, 7, 2, 9, 1, 6, 3};
    for (int i = 0; i < MAX_PROCESSES; i++) {
        procs[i].id       = i + 1;
        procs[i].priority = priorities[i];
        procs[i].active   = true;
    }
}

//       Print current state

void printState() {
    cout << "\nProcess States (Ring order P1->P2->...->P7->P1):" << endl;
    cout << "+---------+----------+--------+" << endl;
    cout << "| Process | Priority | Status |" << endl;
    cout << "+---------+----------+--------+" << endl;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        cout << "|   P" << procs[i].id
             << "    |    " << procs[i].priority
             << "     |   " << (procs[i].active ? "1" : "0")
             << "    |" << endl;
    }
```

```
    cout << "+---------+----------+--------+" << endl;
}

//       Get next active process index (clockwise)

int nextActive(int current) {
    int next = (current + 1) % MAX_PROCESSES;
    while (!procs[next].active && next != current)
        next = (next + 1) % MAX_PROCESSES;
    return next;
}

//       Run Ring Election from initiator processIndex

int ringElection(int initiatorIndex) {
    cout << "\n>> Process P" << procs[initiatorIndex].id
         << " (priority=" << procs[initiatorIndex].priority
         << ") initiates election." << endl;

    vector<int> activeList;
    activeList.push_back(procs[initiatorIndex].priority);

    int current = nextActive(initiatorIndex);

    // Pass message around the ring
    while (current != initiatorIndex) {
        if (procs[current].active) {
            activeList.push_back(procs[current].priority);
            cout << "   ELECTION message passes through P"
                 << procs[current].id
                 << "  active_list: [";
            for (size_t k = 0; k < activeList.size(); k++) {
                cout << activeList[k];
                if (k < activeList.size()-1) cout << ", ";
            }
            cout << "]" << endl;
        }
        current = nextActive(current);
    }

    // Initiator receives its own message back
    cout << "\n>> P" << procs[initiatorIndex].id
         << " received its own message. Active list complete: [";
    for (size_t k = 0; k < activeList.size(); k++) {
        cout << activeList[k];
        if (k < activeList.size()-1) cout << ", ";
    }
    cout << "]" << endl;

    // Find highest priority
    int maxPriority = *max_element(activeList.begin(), activeList.end());

    // Find coordinator index
    int coordinatorIdx = -1;
    for (int i = 0; i < MAX_PROCESSES; i++) {
        if (procs[i].active && procs[i].priority == maxPriority) {
            coordinatorIdx = i;
            break;
```

```
        }
    }

    // Broadcast coordinator
    cout << "\n>> P" << procs[initiatorIndex].id
         << " broadcasts COORDINATOR: P"
         << procs[coordinatorIdx].id << endl;

    cout << "\n==> COORDINATOR elected: P"
         << procs[coordinatorIdx].id
         << " (priority=" << procs[coordinatorIdx].priority << ")"
         << endl;

    return coordinatorIdx;
}

int main() {
    cout << "==========================================" << endl;
    cout << "  Ring Algorithm -- 7 Processes           " << endl;
    cout << "==========================================" << endl;

    initProcesses();
    printState();

    //       Round 1: P3 (index 2) initiates
    cout << "\n--- ROUND 1: Process P3 initiates election ---" << endl;
    int coordinator = ringElection(2);

    //       Coordinator fails
    cout << "\n>> Coordinator P" << procs[coordinator].id
         << " has FAILED. Setting inactive." << endl;
    procs[coordinator].active = false;

    printState();

    //       Round 2: P3 initiates again
    cout << "\n--- ROUND 2: P3 initiates election "
         << "(6 remaining processes) ---" << endl;
    ringElection(2);

    cout << "\n==========================================" << endl;
    cout << "  Simulation Complete" << endl;
    cout << "==========================================" << endl;

    return 0;
}
```

Listing 3: Ring Algorithm – 7 Processes

Output: Ring Algorithm

```
==========================================
  Ring Algorithm -- 7 Processes
==========================================

Process States (Ring: P1->P2->...->P7->P1):
+---------+----------+--------+
| Process | Priority | Status |
+---------+----------+--------+
|   P1    |    4     |   1    |
|   P2    |    7     |   1    |
|   P3    |    2     |   1    |
|   P4    |    9     |   1    |
|   P5    |    1     |   1    |
|   P6    |    6     |   1    |
|   P7    |    3     |   1    |
+---------+----------+--------+

--- ROUND 1: Process P3 initiates election ---

>> Process P3 (priority=2) initiates election.
   ELECTION passes through P4  active_list: [2, 9]
   ELECTION passes through P5  active_list: [2, 9, 1]
   ELECTION passes through P6  active_list: [2, 9, 1, 6]
   ELECTION passes through P7  active_list: [2, 9, 1, 6, 3]
   ELECTION passes through P1  active_list: [2, 9, 1, 6, 3, 4]
   ELECTION passes through P2  active_list: [2, 9, 1, 6, 3, 4, 7]

>> P3 received its own message back. List: [2, 9, 1, 6, 3, 4, 7]

>> P3 broadcasts COORDINATOR: P4

==> COORDINATOR elected: P4 (priority=9)

>> Coordinator P4 has FAILED. Setting inactive.

Process States (Ring: P1->P2->...->P7->P1):
+---------+----------+--------+
| Process | Priority | Status |
+---------+----------+--------+
|   P1    |    4     |   1    |
|   P2    |    7     |   1    |
|   P3    |    2     |   1    |
|   P4    |    9     |   0    |
|   P5    |    1     |   1    |
|   P6    |    6     |   1    |
|   P7    |    3     |   1    |
+---------+----------+--------+

--- ROUND 2: P3 initiates election (6 remaining processes) ---

>> Process P3 (priority=2) initiates election.
   ELECTION passes through P5  active_list: [2, 1]
   ELECTION passes through P6  active_list: [2, 1, 6]
   ELECTION passes through P7  active_list: [2, 1, 6, 3]
   ELECTION passes through P1  active_list: [2, 1, 6, 3, 4]
   ELECTION passes through P2  active_list: [2, 1, 6, 3, 4, 7]
```

```

>> P3 received its own message back. List: [2, 1, 6, 3, 4, 7]

>> P3 broadcasts COORDINATOR: P2

==> COORDINATOR elected: P2 (priority=7)

==========================================
  Simulation Complete
==========================================
```

Listing 4: Ring Algorithm – Actual Output

6. Code Files

- `bully_algorithm.cpp` – Task 1, Bully Algorithm (7 processes)
- `ring_algorithm.cpp` – Task 2, Ring Algorithm (7 processes)

GitHub Repository:
https://github.com/Abrarbut/parallel-and-computing-distribution.git

7. Deliverables

Upload a single Word/PDF file with:

- Source code for both algorithms.
- Screenshots of program output.